МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра технической кибернетики

ЛАБОРАТОРНАЯ РАБОТА № 0

«Введение в MapReduce»

по курсу
Большие данные

Группа 6403
Студент_____ А.А. Кучумов
(*подпись*)

Преподаватель,
Ассистент каф. ТК _____ В.И. Проценко
(*подпись*)

Самара 2025

ЗАДАНИЕ

В вычислимом документе MapReduceExamples.ipynb поэтапно развивается модель MapReduce на языке Python. После каждого этапа приводятся примеры задач, сформулированных в рамках полученной модели. В конце документа вам предлагается решить ряд задач из книги Rajaraman A., Ullman J. D. Mining of massive datasets. – Cambridge University Press, 2011.

ХОД ВЫПОЛНЕНИЯ

В рамках лабораторной работы рассматривается идея конвейера «RecordReader» – «Map» – «Reduce». «RecordReader» имитирует чтение записей с диска или по сети, «Map» выполняет группировку по ключу, «Reduce» осуществляет некоторую агрегацию данных (сумма, максимум, количество). Помимо обычного MapReduce рассматривается распределенный MapReduce.

Для демонстрации указанной выше модели используются различные примеры: SQL-подобный запрос, умножение матрицы на вектор, подсчет слов в строке.

В качестве первого задания, требовалось разработать MapReduce-алгоритм для нахождения максимального числа в списке. На рисунке 1 приведен вывод реализованного решения.

```
[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
[('max', 9)]
```

Рисунок 1 – Решение первого задания

Второе задание заключалось в нахождении среднего значения ряда. На рисунке 2 приведен результат работы для этого задания

```
[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
[('mean', 4.0)]
```

Рисунок 2 – Решение второго задания

В рамках третьего задания необходимо было реализовать groupByKey на основе сортировки. Его работа была проверена на примере и представлена на рисунке 3.

```
Input: [('b', 2), ('a', 1), ('c', 3), ('a', 4), ('b', 5), ('a', 6)]

groupbykey (hash-based):
  ('b', [2, 5])
  ('a', [1, 4, 6])
  ('c', [3])

groupbykey_sort (sort-based):
  ('a', [1, 4, 6])
  ('b', [2, 5])
  ('c', [3])

Same keys: True
Same values (sorted): True
```

Рисунок 3 – Решение третьего задания

Четвертое задание было посвящено реализации распределенной операции исключения дубликатов. Пример работы решения приведен на рисунке 4.

```
13 key-value pairs were sent over a network.
Input:   [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 9, 9, 2, 1]
Unique:  [1, 2, 3, 4, 5, 6, 9]
```

Рисунок 4 – Решение четвертого задания

Пятое, шестое, седьмое, восьмое, девятое, десятое, одиннадцатое и двенадцатое задания были посвящены операциям реляционной алгебры, таким как выборка, проекция, объединение, пересечение, разница, естественное соединение, группировка и агрегация. На рисунках 5, 6, 7, 8, 9, 10, и 11 представлены примеры решений.

```
Employee(id=1, name='Alice', department='Engineering', salary=95000)
Employee(id=2, name='Bob', department='Marketing', salary=60000)
Employee(id=3, name='Carol', department='Engineering', salary=82000)
Employee(id=4, name='Dave', department='HR', salary=55000)
Employee(id=5, name='Eve', department='Engineering', salary=110000)

sigma(department='Engineering' AND salary>85000)(R):
  Employee(id=1, name='Alice', department='Engineering', salary=95000)
  Employee(id=5, name='Eve', department='Engineering', salary=110000)
```

Рисунок 5 – Решение пятого задания

```
Employee(id=1, name='Alice', department='Engineering', salary=95000)
Employee(id=2, name='Bob', department='Marketing', salary=60000)
Employee(id=3, name='Carol', department='Engineering', salary=82000)
Employee(id=4, name='Dave', department='Engineering', salary=82000)
Employee(id=5, name='Carol', department='Engineering', salary=110000)

pi(name, department)(R) - 5 rows - 4 distinct rows:
  EmployeeProjected(name='Alice', department='Engineering')
  EmployeeProjected(name='Bob', department='Marketing')
  EmployeeProjected(name='Carol', department='Engineering')
  EmployeeProjected(name='Dave', department='Engineering')
```

Рисунок 6 – Решение шестого задания

```
Employee(id=1, name='Alice', department='Engineering')
Employee(id=2, name='Bob', department='Marketing')
Employee(id=3, name='Carol', department='Engineering')

Employee(id=3, name='Carol', department='Engineering')
Employee(id=4, name='Dave', department='HR')
Employee(id=5, name='Eve', department='Engineering')

R union S - |R|=3, |S|=3 - 5 distinct tuples:
  Employee(id=1, name='Alice', department='Engineering')
  Employee(id=2, name='Bob', department='Marketing')
  Employee(id=3, name='Carol', department='Engineering')
  Employee(id=4, name='Dave', department='HR')
  Employee(id=5, name='Eve', department='Engineering')
```

Рисунок 7 – Решение седьмого задания

```
Employee(id=1, name='Alice', department='Engineering')
Employee(id=2, name='Bob', department='Marketing')
Employee(id=3, name='Carol', department='Engineering')

Employee(id=3, name='Carol', department='Engineering')
Employee(id=4, name='Dave', department='HR')
Employee(id=5, name='Eve', department='Engineering')

R intersection S - |R|=3, |S|=3 - 1 tuples:
  Employee(id=3, name='Carol', department='Engineering')
```

Рисунок 8 – Решение восьмого задания

```
Employee(id=1, name='Alice', department='Engineering')
Employee(id=2, name='Bob', department='Marketing')
Employee(id=3, name='Carol', department='Engineering')

Employee(id=3, name='Carol', department='Engineering')
Employee(id=4, name='Dave', department='HR')
Employee(id=5, name='Eve', department='Engineering')

R - S - |R|=3, |S|=3 - 2 tuples:
  Employee(id=1, name='Alice', department='Engineering')
  Employee(id=2, name='Bob', department='Marketing')
```

Рисунок 9 – Решение девятого задания

```
('Alice', 10)
('Bob', 20)
('Carol', 10)
('Dave', 30)

(10, 'Engineering')
(20, 'Marketing')

R natural join S on department id:
  name=Alice, dept_id=10, dept=Engineering
  name=Carol, dept_id=10, dept=Engineering
  name=Bob, dept_id=20, dept=Marketing
```

Рисунок 10 – Решение десятого задания

В двенадцатом задании рассматривается случай, когда вектор не помещается в памяти Map-задачи. На рисунке 11 представлен результат работы программы для случайной матрицы и вектора.

```
Employee(name='Carol', department='Engineering', salary=82000)
Employee(name='Dave', department='Marketing', salary=72000)
Employee(name='Eve', department='Engineering', salary=110000)
Employee(name='Frank', department='HR', salary=55000)

gamma(department, SUM(salary)):
  Engineering     287000
  HR              55000
  Marketing       132000

gamma(department, MAX(salary)):
  Engineering     110000
  HR              55000
  Marketing       72000

gamma(department, MIN(salary)):
  Engineering     82000
  HR              55000
  Marketing       60000

gamma(department, COUNT(salary)):
  Engineering     3
  HR              1
  Marketing       2

gamma(department, AVG(salary)):
  Engineering     95666.66666666667
  HR              55000.0
  Marketing       66000.0
```

Рисунок 11 – Решение одиннадцатого задания

```
[[0.32434879 0.24546794 0.08517161 0.93294977 0.30232873 0.37561495]
 [0.61117809 0.30960746 0.29639292 0.06684083 0.989073   0.9278041 ]
 [0.54063048 0.10251755 0.24625015 0.5546831  0.6646675  0.90325329]
 [0.26257514 0.37255588 0.14065113 0.2496532  0.12577401 0.95041005]]

[0.27792889 0.15388055 0.96270606 0.74251604 0.6766007  0.13066351]

Result matches numpy: True
M * v = [1.15627903 1.34291389 1.38269663 0.66036622]
```

Рисунок 12 – Решение двенадцатого задания

В качестве тринадцатого задания предлагалось реализовать умножение матриц в случае, когда одна матрица хранится в памяти, а вторая генерируется «RecordReader»`ом. А в качестве четырнадцатого задания решалась похожая задача, однако обе матрицы генерировались «RecordReader»`ом. На рисунках 13

и 14 приведены перемножаемые матрицы и проверка умножения путем сравнения с результатом работы библиотечной функции.

```
[[0.19657038 0.97000355 0.94543656]
 [0.23839138 0.64105957 0.25298264]]

[[0.35938658 0.84307859 0.53784038 0.61250914 0.93680445 0.79663239
  0.53586482 0.92003442 0.96929563 0.32439997 0.18921414 0.49350018
  0.16407623 0.14168964 0.10207126 0.82845816 0.59893209 0.32470436
  0.86739208 0.56269006 0.25290367 0.69122162 0.66789021 0.58371184
  0.64015672 0.86461016 0.19171663 0.36207572 0.78677264 0.19721811
  0.82619809 0.00106047 0.1506874  0.23008822 0.13812493 0.55467773
  0.08836443 0.50351275 0.71661825 0.38429338]
 [0.2406545  0.69955105 0.26862062 0.73908248 0.77370205 0.75132091
  0.60785633 0.18510612 0.13470341 0.24428104 0.84422495 0.34563971
  0.50669218 0.97558164 0.92662675 0.8029698  0.24206887 0.65934249
  0.41129568 0.95839661 0.29901873 0.68460918 0.91120956 0.42033629
  0.07764613 0.42018763 0.20293576 0.86138965 0.98500728 0.41366435
  0.78283566 0.2696216  0.58370103 0.58929649 0.19897044 0.84118215
  0.96471104 0.47369844 0.6236599  0.73438327]
 [0.30124252 0.22351018 0.9499295  0.87498854 0.07133684 0.87196801
  0.89225645 0.98833651 0.92835205 0.89653273 0.55476632 0.73646048
  0.57186319 0.15979305 0.40608249 0.19768845 0.06156499 0.36038527
  0.22650278 0.68227093 0.64652257 0.93000389 0.26405876 0.0848198
  0.85894948 0.62251428 0.27938109 0.92310809 0.48393865 0.80274123
  0.49052976 0.25648975 0.22499397 0.66955629 0.08426127 0.28055774
  0.90653552 0.11525088 0.28542115 0.27430408]]

True
```

Рисунок 13 – Решение тринадцатого задания

```
[[0.22443111 0.7190511  0.75956736 0.90671973]
 [0.0505582  0.93454725 0.44521713 0.83561637]
 [0.32374161 0.4813037  0.49589149 0.31909923]]

[[0.25460768 0.26784054 0.59318911 0.10173151 0.61663346]
 [0.25037418 0.18763639 0.51754303 0.0694858  0.43502196]
 [0.92785462 0.20609852 0.08618927 0.17741018 0.25736915]
 [0.43140835 0.87964837 0.81912166 0.24543736 0.88532376]]

Result matches numpy: True
```

Рисунок 14 – Решение четырнадцатого задания

В задании №15 предлагалось реализовать умножение матриц с использованием модельного кода «DistributedMapReduce», когда каждая матрица генерируется в своем «RecordReader»`е. На рисунке 15 приведен вывод решения задания №15.

```
32 key-value pairs were sent over a network.
60 key-value pairs were sent over a network.
[[0.21927417 0.81603113 0.30046906 0.91053647]
 [0.56296645 0.57749354 0.28913144 0.65447032]
 [0.57702773 0.49102182 0.71803867 0.98859453]]

[[0.70349737 0.7190886  0.58761274 0.74088973 0.55916767]
 [0.97114871 0.75465526 0.15193224 0.25921321 0.41545517]
 [0.38479677 0.86450735 0.05430333 0.89533296 0.723296  ]
 [0.87483212 0.70434938 0.61852543 0.92379212 0.84445569]]

Result matches numpy: True
```

Рисунок 15 – Решение пятнадцатого задания

И заключительное задание представляет из себя обобщенный случай, когда каждая матрица генерируется несколькими «RecordReader»`ами. На рисунке 16 приведен вывод.

```
50 key-value pairs were sent over a network.
120 key-value pairs were sent over a network.
Test 1 – full matrices, 2 RECORDREADERs for M, 3 for N:
  Matches numpy: True
25 key-value pairs were sent over a network.
28 key-value pairs were sent over a network.

Test 2 - random 50% subsets of M and N:
  Matches full M * N:  False
  Matches partial M * N: True
```

Рисунок 16 – Решение шестнадцатого задания

Все задания реализованы в рамках JupyterNotebook и представлены по ссылке:
https://github.com/artmkchmv/BigData/blob/main/Lab0/6403_KuchumovAA_BigData_Lab_0.ipynb

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы я познакомился с моделью «MapReduce» на Python и убедился, что ей можно решать различные практические задачи (WordCount, инвертированный индекс, агрегация по данным), так и реализовывать операторы реляционной алгебры и умножение матриц в виде соединения по общему индексу с последующей агрегацией.

Помимо прочего я освоил распределённую вариацию с «InputFormat», «Partitioner» и «Combiner».

В упражнениях я реализовал поиск максимума и среднего, groupByKey через сортировку, удаление дубликатов и проверил корректность умножения матриц в разных сценариях.